

Session 9 :

Stream & Pipeline Processing

1. Group Anagrams :

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            char[] chars = s.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```



2. Top K Frequent Elements :

```
import java.util.*;

class Solution {

    public int[] topKFrequent(int[] nums, int k) {

        Map<Integer, Integer> freq = new HashMap<>();

        for (int num : nums) {

            freq.put(num, freq.getOrDefault(num, 0) + 1);

        }

        List<Integer>[] buckets = new List[nums.length + 1];

        for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {

            int num = entry.getKey();

            int count = entry.getValue();

            if (buckets[count] == null) {

                buckets[count] = new ArrayList<>();

            }

        }

    }

}
```

```

        buckets[count].add(num);
    }

    int[] result = new int[k];

    int index = 0;

    for (int i = buckets.length - 1; i >= 0 && index < k; i--) {

        if (buckets[i] != null) {

            for (int num : buckets[i]) {

                result[index++] = num;

                if (index == k) {

                    break;

                }

            }

        }

    }

    return result;

}
}

```

